

CAMS

Computer Account Management System

WRITTEN USE CASES

26 system use cases, with 84 use cases in their diagrams

Admin and Teacher 12 · Admin 5 · Teacher 6 · Student 3

About this document

Each system use case is shown first as a diagram, and its written use case follows. The written use case has ten parts: use case name, purpose, actors, input parameters, output parameters, pre-condition, post-condition, successful scenario, exception scenario and additional remarks.

The admin and the teacher share many use cases, so these are shown only once, under **ADMIN AND TEACHER**. The use cases are joined to Admin, and Teacher points to Admin with a hollow triangle (generalization), so the teacher inherits them.

DELETE is used only for restriction rules, the blacklist and the whitelist, because these are really deleted. Other records are turned on or off with Toggle ... Status, so their records are kept.

«include» means a use case always does the other use case. «extend» means the other use case is optional.

The diagram is in CAMS-Use-Case-Diagram.drawio, in the same folder.

Contents

About this document.....	2
ADMIN AND TEACHER	5
Process Log In	5
Process Sign Out	8
Manage Own Account	10
Manage Teacher Account.....	12
Manage Student Account	15

Manage Computer Profile.....	18
Manage Class.....	21
Manage Restriction Rule.....	24
Manage Blacklist and Whitelist.....	27
Manage Category	30
Manage Session Rule	33
Control Laboratory Session	36
ADMIN.....	39
Manage Admin Account.....	39
Manage Class Teacher.....	42
Export Reports and Logs	44
Manage Database	46
Manage Deployment	49
TEACHER.....	51
Control Student Session	51
Monitor Student Screen	54
Control Student Workstation	56
Send Message to Student.....	59
Manage Monitoring Alert	61
Export Teacher Records.....	63

STUDENT	65
Log In at Workstation	65
Log Out at Workstation	68
Change Password at Workstation	70

ADMIN AND TEACHER

The admin and the teacher can both do the use cases in these system use cases. The use cases are joined to Admin, and the line with a hollow triangle from Teacher to Admin is a generalization: the teacher inherits the admin's use cases here.

A use case only one of them has is in that actor's own section: Assign Class Teacher is under ADMIN (Manage Class Teacher), and Start Lab Session is under TEACHER (Control Student Session).

Process Log In

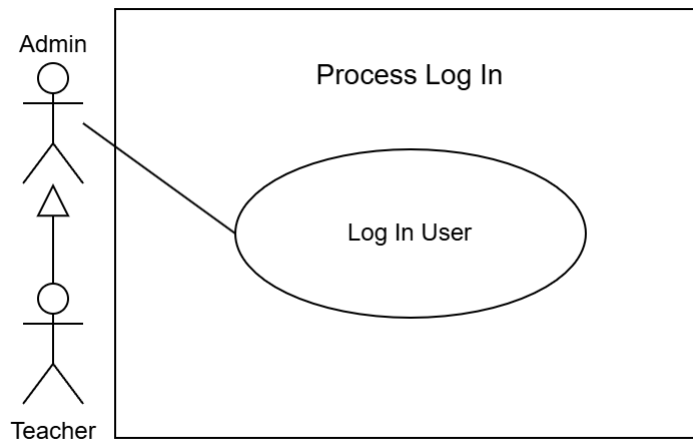


Figure 3.1: System Use Case for Process Log In

Use Case Name: Process Log In

Purpose: To allow the admin or teacher to log in to the CAMS website using their username and password.

Actors: Admin, Teacher

Input Parameters:

- Username

- Password

Output Parameters:

- Login success or error message
- The dashboard for the user's role

Pre-Condition: The user has an active account in CAMS.

Post-Condition: The user is logged in and sees the Admin or Teacher dashboard.

Successful Scenario:

1. The user opens the CAMS login page.
2. The user enters the username and password.
3. The user clicks Sign in.
4. The system checks the username and password (Log In User):
 - If valid, the system opens the Admin or Teacher dashboard.
 - If not valid, the system shows an error and the user can try again.

Exception Scenario:

- If the user enters a wrong password 5 times:
 - The system locks the account for 15 minutes.
- If the account is turned off (inactive):
 - The system does not let the user log in.
- If a student tries to log in on the website:
 - The system tells the student to use the CAMS student app on the lab computer.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- A locked teacher account can be unlocked early on the Teachers page (Manage Teacher Account).

Process Sign Out

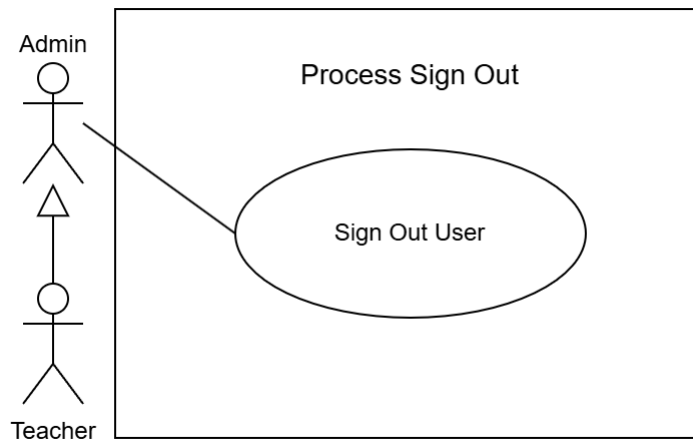


Figure 3.2: System Use Case for Process Sign Out

Use Case Name: Process Sign Out

Purpose: To allow the admin or teacher to sign out of the CAMS website when they are done.

Actors: Admin, Teacher

Input Parameters: None (the user clicks Sign out)

Output Parameters: The login page

Pre-Condition: The user is logged in.

Post-Condition: The user is signed out, and the next person must log in again.

Successful Scenario:

1. The user clicks Sign out (Sign Out User).
2. The system ends the user's session.
3. The system shows the login page.

Exception Scenario:

- If the session has already ended:
 - The system just shows the login page.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- Signing out is separate from logging in (Process Log In).

Manage Own Account

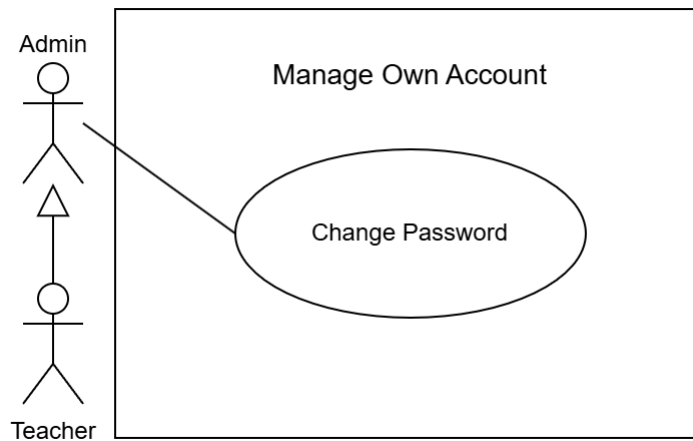


Figure 3.3: System Use Case for Manage Own Account

Use Case Name: Manage Own Account

Purpose: To allow the admin or teacher to change their own password.

Actors: Admin, Teacher

Input Parameters:

- Current password
- New password
- New password again, to confirm

Output Parameters: A message that the password was changed

Pre-Condition: The user is logged in.

Post-Condition: The new password is saved and must be used at the next login.

Successful Scenario:

1. The user opens the Settings page.
2. The user enters the current password and the new password twice.

3. The user clicks Change password (Change Password).
4. The system checks the passwords:
 - If correct, the system saves the new password and shows a message.

Exception Scenario:

- If the current password is wrong:
 - The system shows an error and keeps the old password.
- If the new password has fewer than 8 characters, or the two new passwords do not match:
 - The system asks the user to enter them again.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- The password is saved in a protected form, not as plain text.

Manage Teacher Account

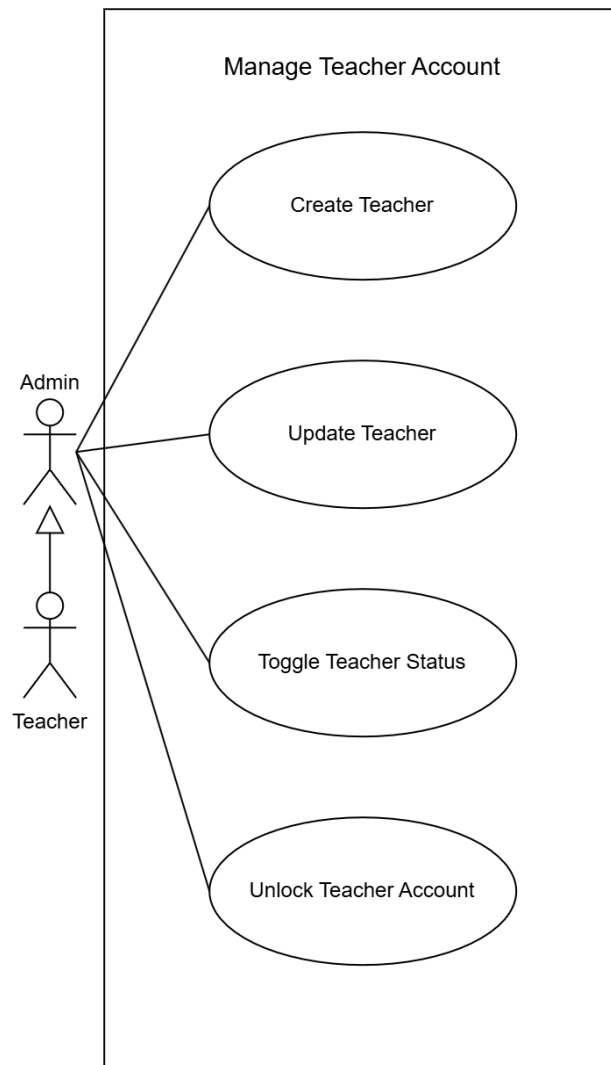


Figure 3.4: System Use Case for Manage Teacher Account

Use Case Name: Manage Teacher Account

Purpose: To allow the user to add, edit, turn on or off, and unlock teacher accounts.

Actors: Admin, Teacher

Input Parameters:

- Teacher details: name, email, contact number, username and password

- The teacher account to edit, turn on or off, or unlock

Output Parameters:

- The updated teacher list
- A success or error message

Pre-Condition:

- The user is logged in.
- To edit, turn off or unlock an account, the account must exist.

Post-Condition:

- The teacher account is added, updated, turned on or off, or unlocked.
- The change is saved in the audit trail.

Successful Scenario:

1. The user opens the Teachers page.
2. The user chooses what to do:
 - Create Teacher: clicks Add, fills in the details and clicks Save.
 - Update Teacher: clicks Edit, changes the details and clicks Save.
 - Toggle Teacher Status: clicks Deactivate or Activate and confirms.
 - Unlock Teacher Account: clicks Unlock on a locked account.
3. The system checks the details and saves the change.
4. The system shows the updated teacher list.

Exception Scenario:

- If the username or password is missing, or the username is already used:
 - The system does not save and shows an error.
- If the teacher still has active classes, or is the last active teacher:
 - The system does not turn the account off.
- If a teacher tries to edit, turn off or unlock their own account on this page:
 - The system does not allow it.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- An inactive teacher cannot log in, but the account and its records are kept.
Nothing is deleted.
- A locked account also unlocks by itself after 15 minutes.

Manage Student Account

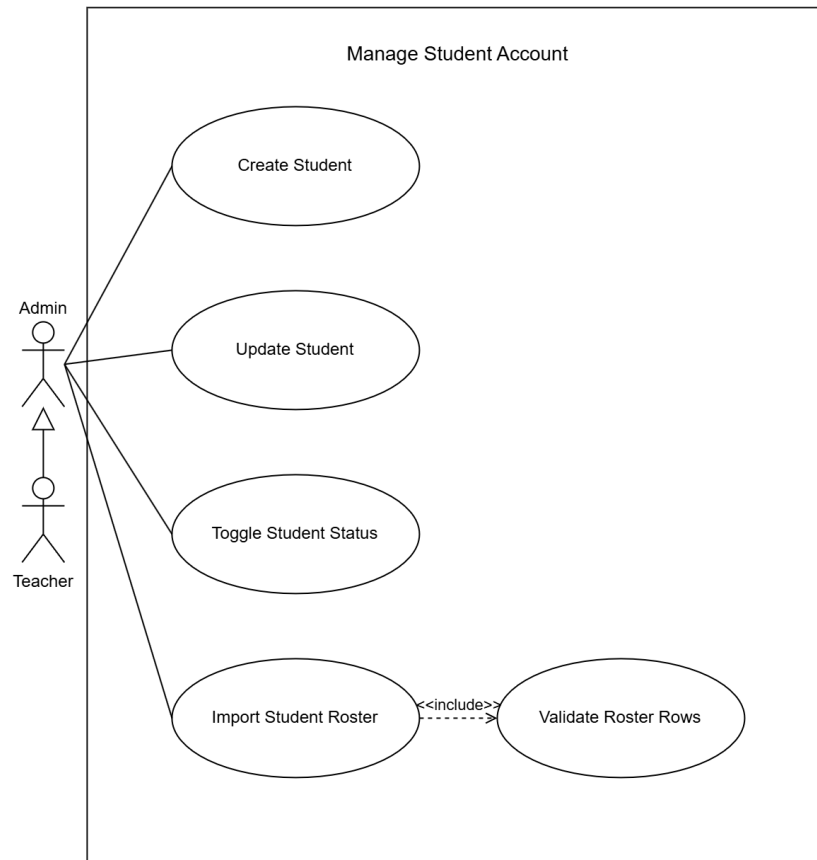


Figure 3.5: System Use Case for Manage Student Account

Use Case Name: Manage Student Account

Purpose: To allow the user to add, edit, and turn on or off student accounts, one at a time or many at once.

Actors: Admin, Teacher

Input Parameters:

- Student details: student number, first name, last name, username and password
- A list of students, typed in or uploaded as a CSV file (for an import)

Output Parameters:

- The updated student list
- A list of the wrong rows, if an import fails

Pre-Condition:

- The user is logged in.
- A class that receives imported students must be active and have a teacher.

Post-Condition:

- The student accounts are added or updated, or turned on or off.
- Active students can log in on a lab computer.

Successful Scenario:

1. The user opens the Students page.
2. The user chooses what to do:
 - Create Student: clicks Add, fills in the details and clicks Save.
 - Update Student: clicks Edit, changes the details and clicks Save.
 - Toggle Student Status: clicks Deactivate or Activate and confirms.
 - Import Student Roster: types the students or uploads a CSV file and clicks Save.
3. For an import, the system first checks every row (Validate Roster Rows).
4. The system saves the students and shows the updated list.

Exception Scenario:

- If a required detail is missing, or the student number or username is already used:
 - The system does not save and shows an error.
- If any row of the import is wrong (no name, a password shorter than 8 characters, or a username used twice):
 - No student is added, and the system shows the wrong rows.
- If the import list is empty:
 - The system asks for at least one student.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- Import Student Roster always includes Validate Roster Rows.
- The Remove button only hides a student. The records are kept and can be restored.

Manage Computer Profile

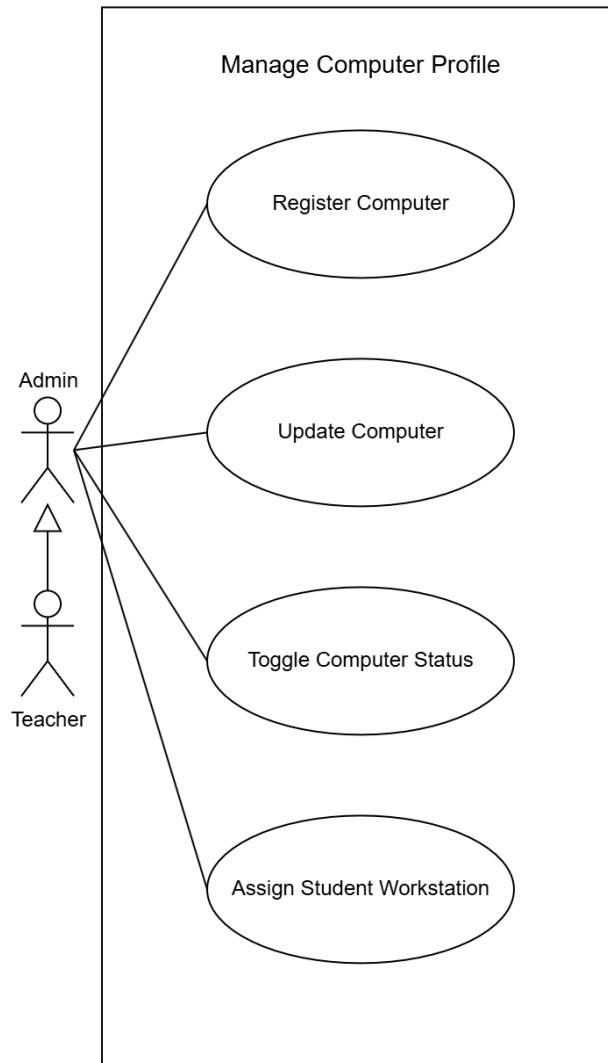


Figure 3.6: System Use Case for Manage Computer Profile

Use Case Name: Manage Computer Profile

Purpose: To allow the user to add lab computers, change their details, archive or bring them back, and reserve a computer for a student.

Actors: Admin, Teacher

Input Parameters:

- Computer details: station name and status
- The student and the computer to reserve

Output Parameters:

- The updated computer list
- A success or error message

Pre-Condition:

- The user is logged in.
- To edit, archive or reserve a computer, it must exist.

Post-Condition:

- The computer is added, updated, archived or brought back, or reserved for the student.
- Status changes are kept in the computer's history.

Successful Scenario:

1. The user opens the Computers page, or the Students page to reserve a computer.
2. The user chooses what to do:
 - Register Computer: clicks Add, fills in the details and clicks Save.
 - Update Computer: clicks Edit, changes the name or status and clicks Save.
 - Toggle Computer Status: clicks Archive; to bring it back, edits it and sets the status to Available.

- Assign Student Workstation: picks a computer for the student and clicks Save.
- 3. The system checks the details and saves the change.
- 4. The system shows the updated list.

Exception Scenario:

- If the station name is missing or already used:
 - The system does not save.
- If a lab session is running on the computer:
 - The system does not archive it.
- If the computer is archived, already reserved or in use:
 - The system does not reserve it for the student.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- An archived computer is hidden from the active list. Nothing is deleted.
- Choosing no computer clears a student's reservation.

Manage Class

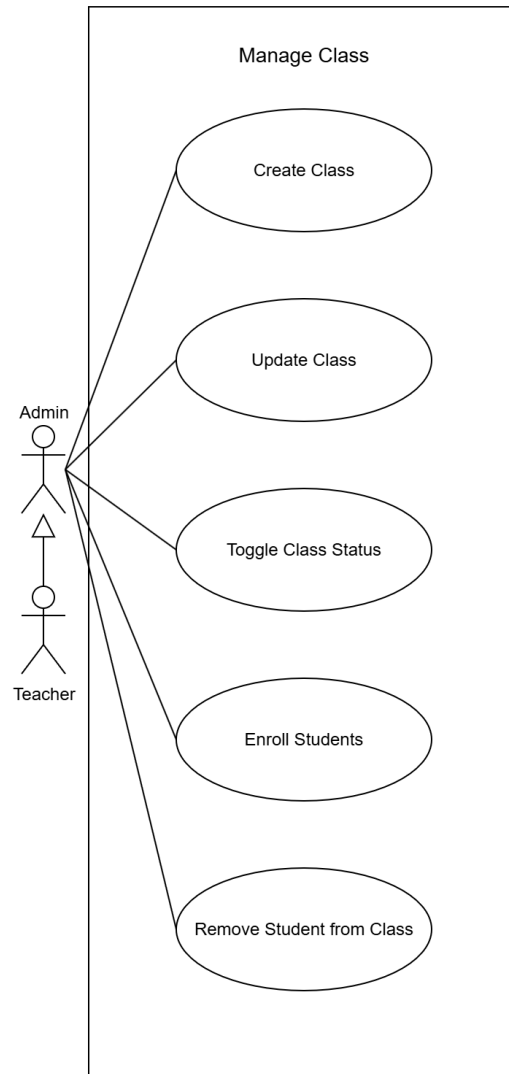


Figure 3.7: System Use Case for Manage Class

Use Case Name: Manage Class

Purpose: To allow the user to create, edit, archive and restore classes, and to put students in or out of a class.

Actors: Admin, Teacher

Input Parameters:

- Class details: class name, grade level, section, subject, schedule and school year
- The students to enroll or remove

Output Parameters:

- The updated class list or class page
- A success or error message

Pre-Condition:

- The user is logged in.
- To enroll students, the class must be active and have a teacher.

Post-Condition:

- The class is added, updated, archived or restored.
- The class has the chosen students.

Successful Scenario:

1. The user opens the Classes page.
2. The user chooses what to do:
 - Create Class: clicks Add, fills in the details and clicks Save.
 - Update Class: clicks Edit, changes the details and clicks Save.
 - Toggle Class Status: clicks Archive or Restore and confirms.
 - Enroll Students: opens the class, picks students (or types a new student) and clicks Enroll.

- Remove Student from Class: opens the class, clicks Remove on the students and confirms.
- 3. The system checks the details and saves the change.
- 4. The system shows the updated class.

Exception Scenario:

- If the class name is missing, or the same class already exists for that school year:
 - The system does not save.
- If a teacher tries to change a class that is not theirs:
 - The system does not allow it.
- If a student is already in another class:
 - The system asks the user to confirm the move.
- If a class to be restored has no active teacher:
 - The system does not restore it.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- A class made by a teacher belongs to that teacher.
- Choosing the teacher of a class is for the admin only (Manage Class Teacher).
- Archiving a class or removing a student keeps all records. Nothing is deleted.

Manage Restriction Rule

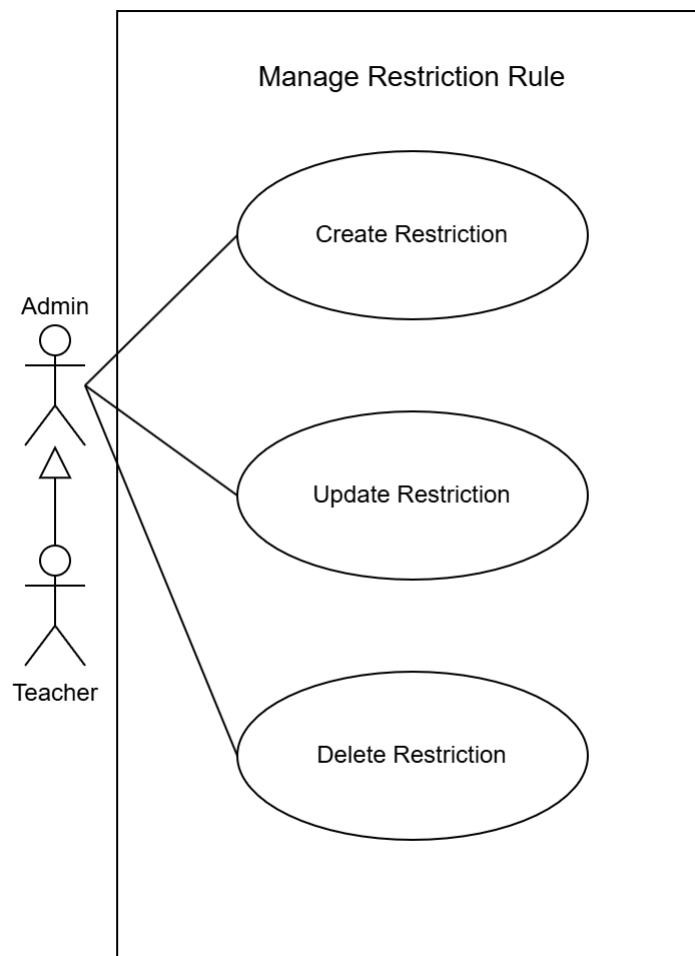


Figure 3.8: System Use Case for Manage Restriction Rule

Use Case Name: Manage Restriction Rule

Purpose: To allow the user to add, change and delete rules that block or allow websites during lab sessions.

Actors: Admin, Teacher

Input Parameters:

- Rule details: rule type, target (website or app), block or allow, description and active

- The rule to change or delete

Output Parameters:

- The updated rule list
- A success or error message

Pre-Condition:

- The user is logged in.
- To change or delete a rule, the rule must exist.

Post-Condition: Student computers follow the new or changed rule, and stop using a deleted rule.

Successful Scenario:

1. The user opens the Restriction Rules page.
2. The user chooses what to do:
 - Create Restriction: clicks Add, fills in the rule and clicks Save.
 - Update Restriction: clicks Edit, changes the rule or turns it on or off, and clicks Save.
 - Delete Restriction: clicks Delete on the rule and confirms.
3. The system checks the details and saves or deletes the rule.
4. The system shows the updated rule list.

Exception Scenario:

- If the rule type, block or allow, or the target is missing:
 - The system does not save.

- If a teacher tries to change or delete a rule that is not theirs:
 - The system does not allow it.
- If the rule is already gone:
 - Nothing changes.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- Websites are blocked. Apps are only watched, never closed.
- A deleted rule cannot be brought back. A rule that is turned off is kept but ignored.

Manage Blacklist and Whitelist

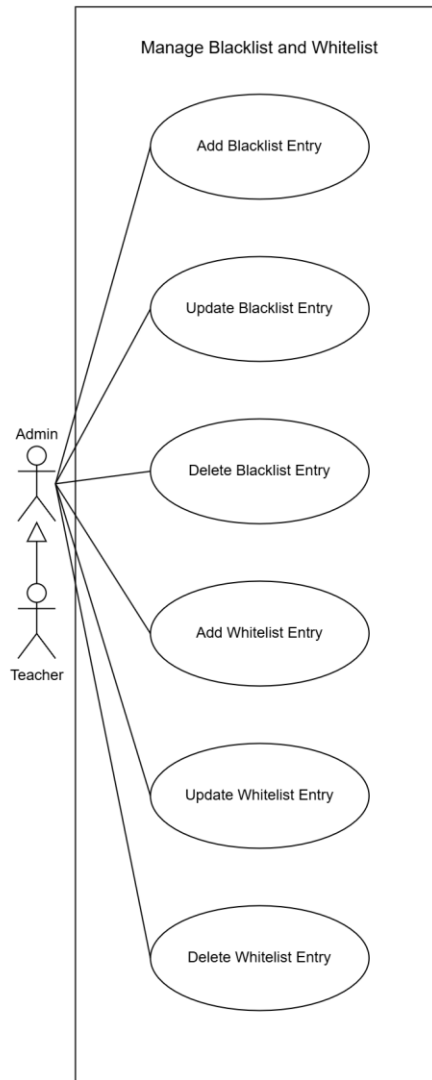


Figure 3.9: System Use Case for Manage Blacklist and Whitelist

Use Case Name: Manage Blacklist and Whitelist

Purpose: To allow the user to keep the list of blocked websites and apps (blacklist) and the list of allowed websites (whitelist).

Actors: Admin, Teacher

Input Parameters:

- Blacklist entry: type (website, domain, app or process), value and reason
- Whitelist entry: website and description
- Active (on or off), when editing an entry

Output Parameters: The updated blacklist or whitelist

Pre-Condition:

- The user is logged in.
- To edit or delete an entry, the entry must exist.

Post-Condition: Student computers block what is on the blacklist and follow the whitelist.

Successful Scenario:

1. The user opens the Blacklist page or the Whitelist page.
2. The user chooses what to do:
 - Add Blacklist Entry or Add Whitelist Entry: clicks Add, fills in the entry and clicks Save.
 - Update Blacklist Entry or Update Whitelist Entry: clicks Edit, changes the entry or turns it on or off, and clicks Save.
 - Delete Blacklist Entry or Delete Whitelist Entry: clicks Delete and confirms.
3. The system checks the details and saves or deletes the entry.
4. The system shows the updated list.

Exception Scenario:

- If the type, the value or the website is missing:
 - The system does not save.
- If the entry is already gone:
 - Nothing changes.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- A blacklisted website stays blocked even if it is on the whitelist.
- Apps on the blacklist are only watched, not closed.
- A deleted entry cannot be brought back.

Manage Category

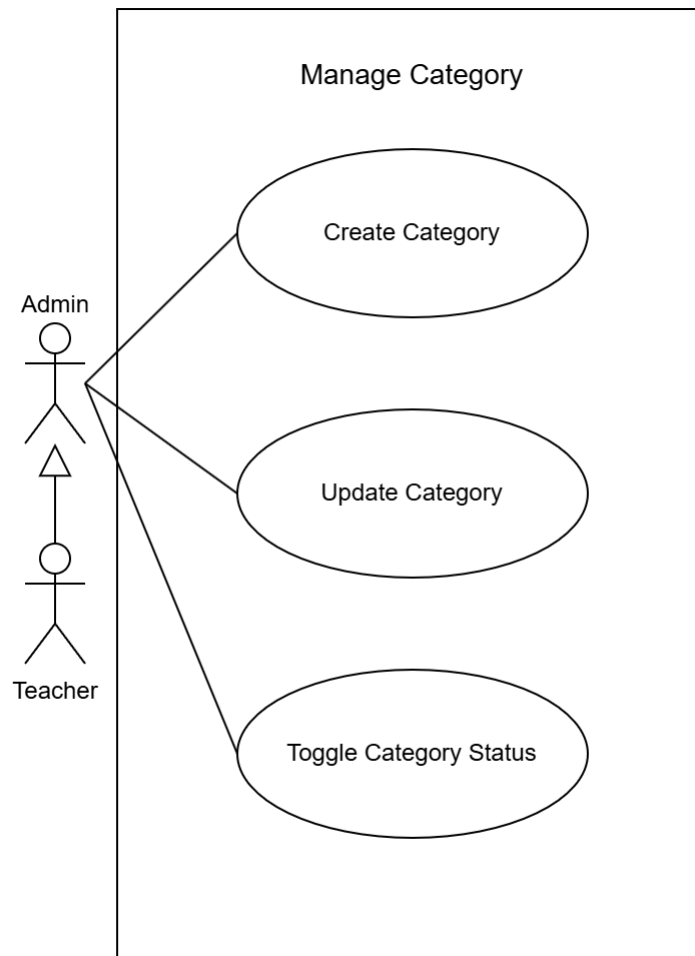


Figure 3.10: System Use Case for Manage Category

Use Case Name: Manage Category

Purpose: To allow the user to group apps or websites, for example all games, so one rule covers them all.

Actors: Admin, Teacher

Input Parameters:

- Category details: name, pattern (app name or website address), block or allow, and description

- Active (on or off)

Output Parameters: The updated category list

Pre-Condition:

- The user is logged in.
- To edit or turn off a category, it must exist.

Post-Condition: Apps or websites that match a category follow it. A category that is off is ignored.

Successful Scenario:

1. The user opens the Restriction Rules page.
2. The user chooses what to do:
 - Create Category: clicks Add Category, fills in the details and clicks Save.
 - Update Category: clicks Edit on the category, changes the details and clicks Save.
 - Toggle Category Status: clicks Edit on the category, turns Active on or off and clicks Save.
3. The system checks the details and saves the category.
4. The system shows the updated categories.

Exception Scenario:

- If the name or the pattern is missing:
 - The system does not save.

- If the category is already gone:
 - Nothing changes.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- There are app categories and website categories.
- A category is turned off, not deleted.

Manage Session Rule

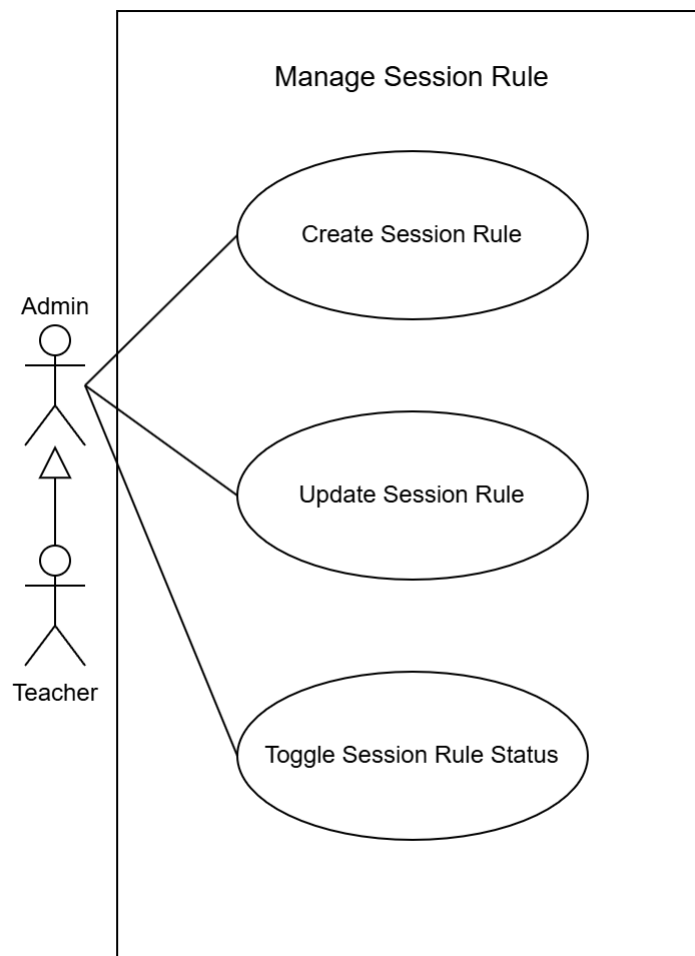


Figure 3.11: System Use Case for Manage Session Rule

Use Case Name: Manage Session Rule

Purpose: To allow the user to set how lab sessions work: the time limit, pausing and remote control.

Actors: Admin, Teacher

Input Parameters: Rule details: name, time limit, allow pause, allow remote control and default

Output Parameters: The updated session rule list

Pre-Condition:

- The user is logged in.
- To edit or turn off a rule, the rule must exist.

Post-Condition: New lab sessions follow the rule. A rule that is off cannot be used for new sessions.

Successful Scenario:

1. The user opens the Session Rules page.
2. The user chooses what to do:
 - Create Session Rule: clicks Add, fills in the details and clicks Save.
 - Update Session Rule: clicks Edit, changes the details and clicks Save.
 - Toggle Session Rule Status: clicks Deactivate and confirms, or edits the rule and turns Active on.
3. The system checks the details and saves the rule.
4. The system shows the updated rule list.

Exception Scenario:

- If the name is missing:
 - The system does not save.
- If the rule is already gone:
 - Nothing changes.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).
- A new default rule replaces the old default.
- Old sessions keep their rule. Nothing is deleted.

Control Laboratory Session

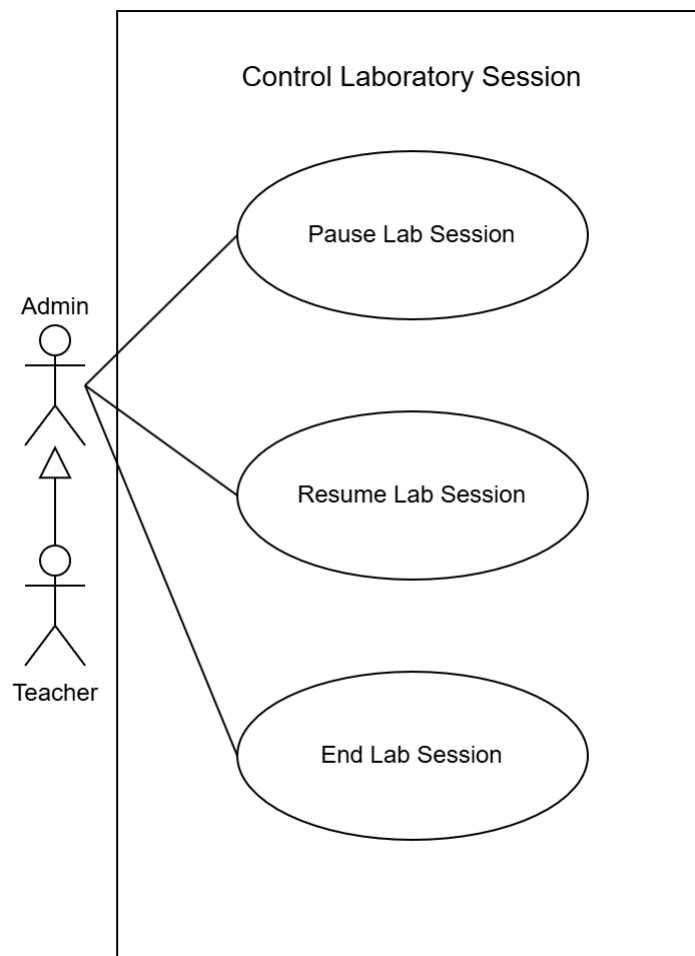


Figure 3.12: System Use Case for Control Laboratory Session

Use Case Name: Control Laboratory Session

Purpose: To allow the user to pause, resume or end all the student sessions in the lab at once.

Actors: Admin, Teacher

Input Parameters: None (the user clicks Pause, Resume or End)

Output Parameters: The number of sessions paused, resumed or ended

Pre-Condition:

- The user is logged in.
- A lab session is running (to pause or end it) or paused (to resume it).

Post-Condition:

- All the sessions are paused, running again, or ended.
- The change is saved in the audit trail.

Successful Scenario:

1. The user opens the dashboard (admin) or the Sessions page (teacher).
2. The user chooses what to do:
 - Pause Lab Session: clicks Pause. The student computers show a pause screen and the timers stop.
 - Resume Lab Session: clicks Resume. The pause screens go away and the timers continue.
 - End Lab Session: clicks End and confirms. The sessions end and the student computers restart.
3. The system applies the change to every session in the lab.
4. The system shows how many sessions were changed.

Exception Scenario:

- If no session is running, paused or open:
 - Nothing changes and the system shows 0 sessions.

Additional Remarks:

- The Teacher inherits this use case from the Admin (generalization).

- Only the teacher can start a lab session (Control Student Session).
- Paused time is not counted, and the session records are kept.

ADMIN

Manage Admin Account

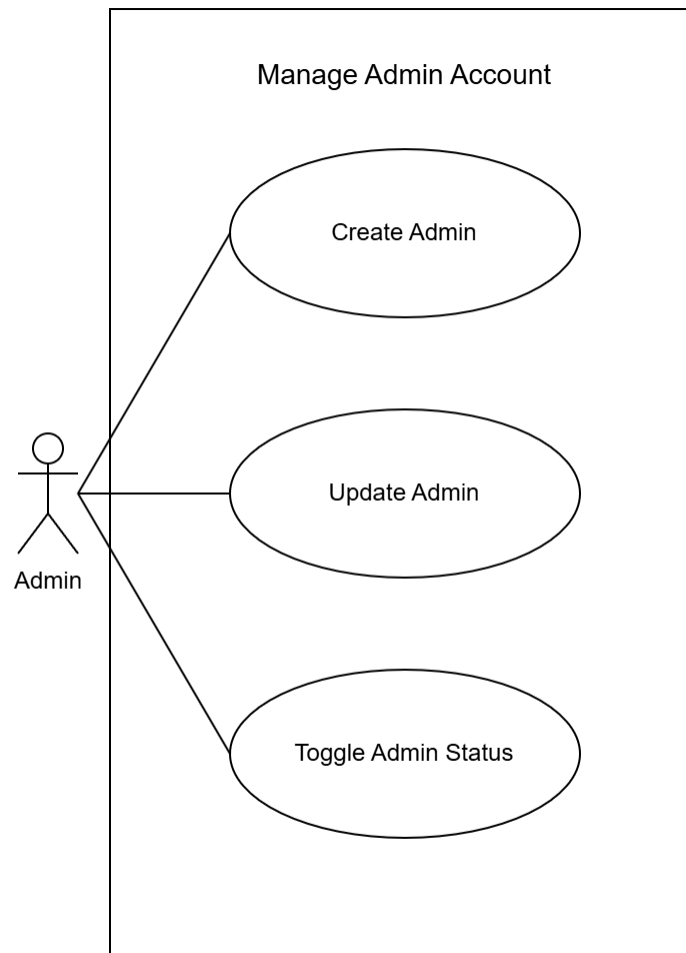


Figure 3.13: System Use Case for Manage Admin Account

Use Case Name: Manage Admin Account

Purpose: To allow the admin to add, edit, and turn on or off admin accounts.

Actors: Admin

Input Parameters:

- Admin details: full name, username and password

- The admin account to edit or turn on or off

Output Parameters: The updated admin list

Pre-Condition: The admin is logged in.

Post-Condition:

- The admin account is added, updated, or turned on or off.
- The change is saved in the audit trail.

Successful Scenario:

1. The admin opens the Admin Accounts page.
2. The admin chooses what to do:
 - Create Admin: clicks Add, fills in the details and clicks Save.
 - Update Admin: clicks Edit, changes the name or username and clicks Save.
 - Toggle Admin Status: clicks Deactivate or Activate and confirms.
3. The system checks the details and saves the change.
4. The system shows the updated list.

Exception Scenario:

- If the username or password is missing, or the username is already used:
 - The system does not save.
- If the admin tries to turn off the last active admin:
 - The system does not allow it.

Additional Remarks:

- An inactive admin cannot log in, but the account is kept. Nothing is deleted.

Manage Class Teacher

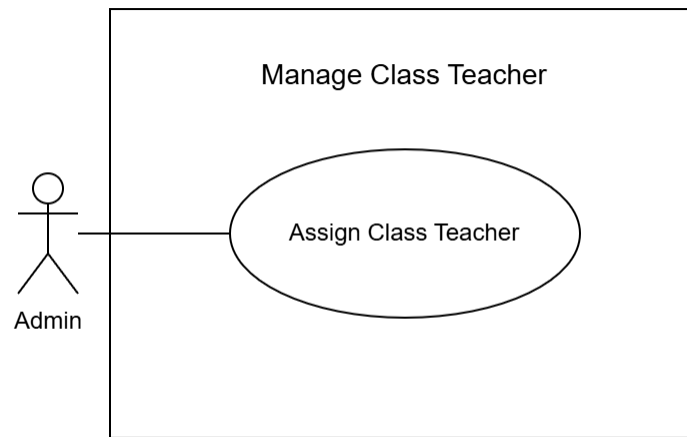


Figure 3.14: System Use Case for Manage Class Teacher

Use Case Name: Manage Class Teacher

Purpose: To allow the admin to choose the teacher in charge of a class, change the teacher, or leave the class without one.

Actors: Admin

Input Parameters:

- The class
- The teacher, or none to leave the class without a teacher

Output Parameters:

- The class page showing its teacher
- A success or error message

Pre-Condition:

- The admin is logged in.
- The class exists.

Post-Condition:

- The chosen teacher is in charge of the class, and the class appears on that teacher's class list.
- The change is saved in the audit trail.

Successful Scenario:

1. The admin opens the Classes page and opens the class.
2. The admin picks a teacher for the class (Assign Class Teacher).
3. The admin clicks Save.
4. The system checks the teacher:
 - If the teacher is active, the system saves the teacher for the class and shows the class page.

Exception Scenario:

- If the chosen teacher is inactive:
 - The system does not assign the class and asks for an active teacher.
- If the class is not found:
 - The system shows an error and nothing changes.

Additional Remarks:

- Only the admin can do this. A teacher who tries is told that only administrators can assign or reassign teachers.

Export Reports and Logs

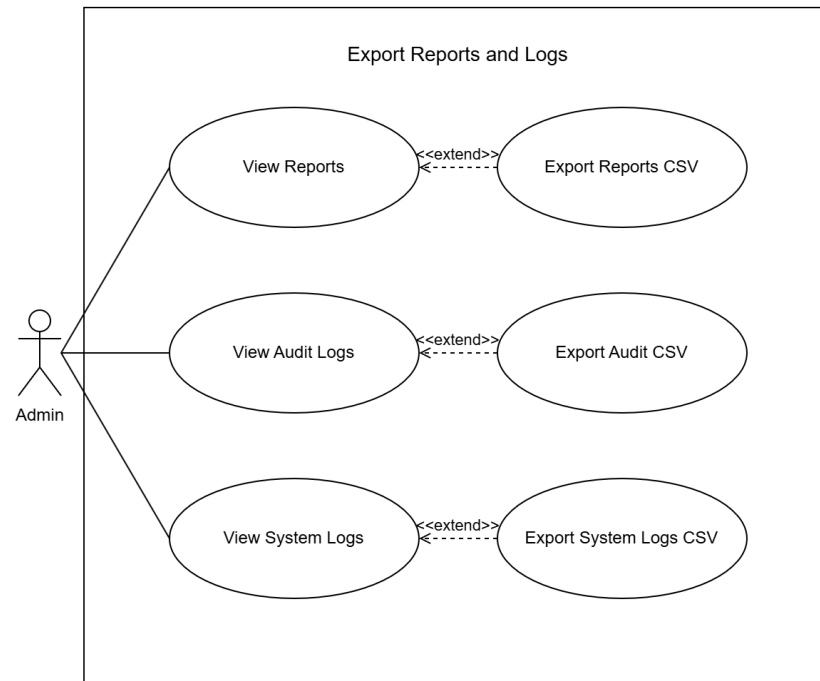


Figure 3.15: System Use Case for Export Reports and Logs

Use Case Name: Export Reports and Logs

Purpose: To allow the admin to see the lab reports, the audit trail and the system logs, and to download them as CSV files.

Actors: Admin

Input Parameters: Filters: date range, and class or computer (optional, for reports)

Output Parameters:

- The Reports, Audit Trail or System Logs page
- A CSV file, when the admin exports

Pre-Condition: The admin is logged in.

Post-Condition: The admin sees the records, and has the CSV file if one was exported.
Nothing is changed.

Successful Scenario:

1. The admin opens the Reports page (View Reports), the Audit Trail page (View Audit Logs) or the System Logs page (View System Logs).
2. The admin sets the filters (optional).
3. The system shows the records.
4. If the admin needs a file, the admin clicks the export button on the page:
 - Export Reports CSV, Export Audit CSV or Export System Logs CSV.
5. The system makes the CSV file and the browser downloads it.

Exception Scenario:

- If nothing matches the filters:
 - The page shows an empty list, and the CSV file has only the column names.

Additional Remarks:

- Each export extends its page: it is optional and happens only while the page is open.
- The Audit Trail and System Logs pages show the latest 500 records.

Manage Database

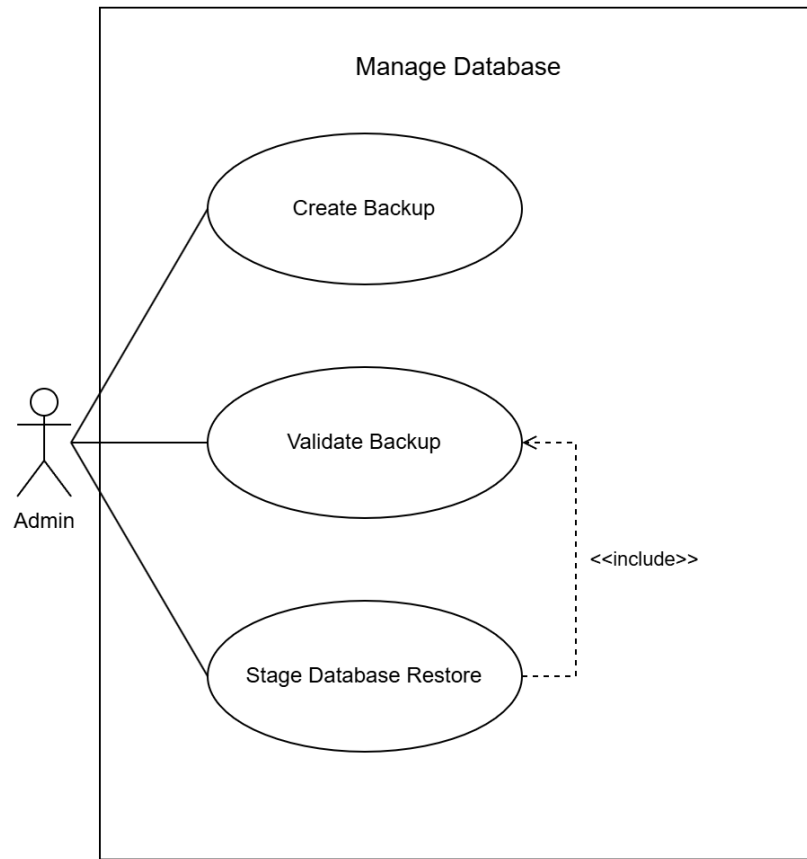


Figure 3.16: System Use Case for Manage Database

Use Case Name: Manage Database

Purpose: To allow the admin to back up the CAMS database, check a backup, and restore the database from a backup.

Actors: Admin

Input Parameters:

- A label for the backup (optional)
- The backup to check or restore
- The word RESTORE, to confirm a restore

Output Parameters:

- The backup list
- A message saying whether the backup passed the check
- A message to restart the server after a restore is set

Pre-Condition:

- The admin is logged in.
- To check or restore a backup, it must be in the list.

Post-Condition: A checked backup is saved, or the chosen backup will replace the database when the server restarts.

Successful Scenario:

1. The admin opens the Database page.
2. The admin chooses what to do:
 - Create Backup: types a label (optional) and clicks Create backup.
 - Validate Backup: clicks Validate on a backup.
 - Stage Database Restore: clicks Restore on a backup and types RESTORE to confirm.
3. For a restore, the system first checks the backup (Validate Backup) and makes a safety copy of the current database.
4. The system does the action and shows the result.

Exception Scenario:

- If making the backup fails:
 - The system shows an error and the database is not changed.

- If the backup is damaged, or RESTORE is not typed:
 - The system does not restore, and nothing is changed.

Additional Remarks:

- Stage Database Restore always includes Validate Backup.
- The restore takes effect only after the server restarts.

Manage Deployment

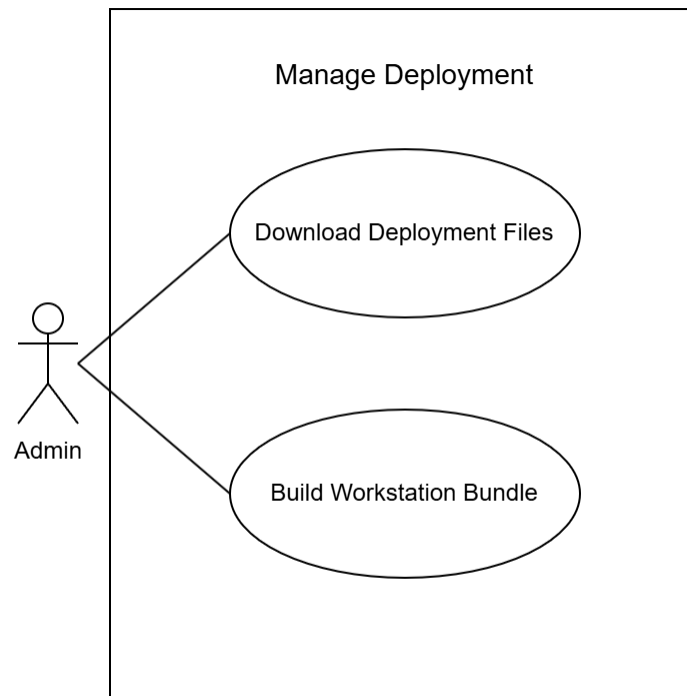


Figure 3.17: System Use Case for Manage Deployment

Use Case Name: Manage Deployment

Purpose: To allow the admin to get the files needed to install the CAMS student app on the lab computers.

Actors: Admin

Input Parameters:

- The file to download: installer, manifest or certificate
- The server address, for a workstation bundle

Output Parameters: The downloaded file, or a zip file (bundle)

Pre-Condition: The admin is logged in.

Post-Condition: The admin has the setup files. Nothing in CAMS is changed.

Successful Scenario:

1. The admin opens the Deployment page.
2. The admin chooses what to do:
 - Download Deployment Files: clicks the installer, the manifest or the certificate.
 - Build Workstation Bundle: enters the server address and clicks Build.
3. The system prepares the file.
4. The browser downloads it.

Exception Scenario:

- If a file is missing or damaged:
 - The download fails and the system shows an error.
- If the server address is wrong:
 - The system does not build the bundle and shows an error.

Additional Remarks:

- The bundle has the installer, the certificate and an install script.
- The certificate lets the lab computers connect to the server safely.

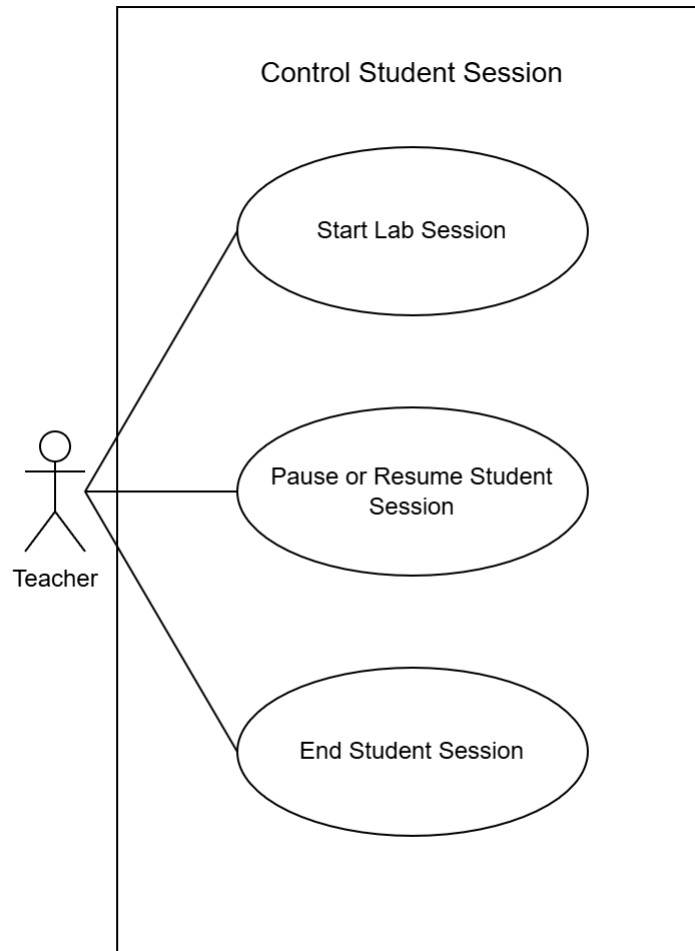
TEACHER**Control Student Session**

Figure 3.18: System Use Case for Control Student Session

Use Case Name: Control Student Session

Purpose: To allow the teacher to start the lab session, and to pause, resume or end one student's session.

Actors: Teacher

Input Parameters:

- Session rule (optional, when starting the lab session)
- The student's session, to pause, resume or end

Output Parameters: A message that the lab session started, or the session's new status

Pre-Condition:

- The teacher is logged in.
- To pause, resume or end a student's session, the session is open.

Post-Condition: Students can log in on any lab computer and join the lab session, or the student's session is paused, running again, or ended.

Successful Scenario:

1. The teacher opens the Sessions page.
2. The teacher chooses what to do:
 - Start Lab Session: clicks Start lab session and picks a session rule, or keeps the default.
 - Pause or Resume Student Session: clicks Pause or Resume on a student's session.
 - End Student Session: clicks End on a student's session and confirms.
3. The system saves the change.
4. When a session ends, the system saves the end time and restarts the student's computer.

Exception Scenario:

- If the chosen session rule is turned off:
 - The system does not start the lab session.
- If the session rule does not allow pausing:
 - The system does not pause the session.
- If the session has already ended:
 - Nothing changes.

Additional Remarks:

- Only the teacher can start a lab session. The admin can only pause, resume or end all sessions (Control Laboratory Session).
- Paused time is not counted, and the session records are kept.

Monitor Student Screen

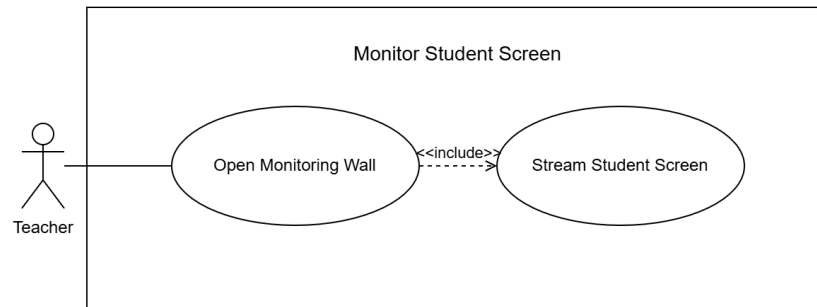


Figure 3.19: System Use Case for Monitor Student Screen

Use Case Name: Monitor Student Screen

Purpose: To allow the teacher to watch all the student screens live on one page.

Actors: Teacher

Input Parameters: None (the teacher opens the Live Monitoring page)

Output Parameters: The live screen of every logged-in student

Pre-Condition:

- The teacher is logged in.
- Students are logged in on the lab computers.

Post-Condition: The teacher sees the latest screens. Nothing is saved.

Successful Scenario:

1. The teacher opens the Live Monitoring page (Open Monitoring Wall).
2. Each student computer sends its screen to the system (Stream Student Screen).
3. The system shows each screen on the page and keeps it updated.

Exception Scenario:

- If no student is logged in:
 - The page shows no screens.
- If a screen picture is empty or too large:
 - The system skips that picture.

Additional Remarks:

- Open Monitoring Wall always includes Stream Student Screen.
- From a screen on this page, the teacher can control that computer (Control Student Workstation).

Control Student Workstation

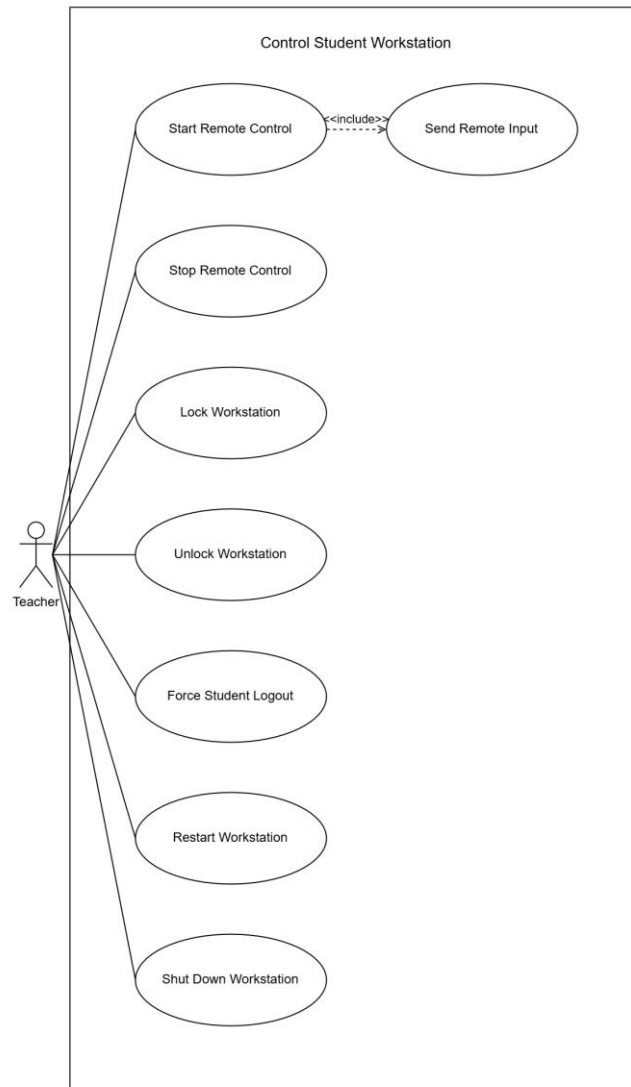


Figure 3.20: System Use Case for Control Student Workstation

Use Case Name: Control Student Workstation

Purpose: To allow the teacher to take control of a student's computer, lock or unlock it, log the student out, or restart or shut down the computer.

Actors: Teacher

Input Parameters:

- One student's computer, or all the computers shown (for lock and log out)
- The teacher's mouse clicks and key presses, during remote control

Output Parameters: The result of the command on the monitoring page

Pre-Condition:

- The teacher is logged in and the computer is connected.
- For remote control, the session rule must allow it.

Post-Condition:

- The student's computer does the command.
- The command is saved in the remote history.

Successful Scenario:

1. The teacher opens the Live Monitoring page and picks a computer, or all the computers shown.
2. The teacher chooses what to do:
 - Start Remote Control: clicks Start Remote Support. The teacher's mouse and keyboard now work on the student's computer (Send Remote Input).
 - Stop Remote Control: clicks Stop Remote Support.
 - Lock Workstation or Unlock Workstation: clicks Lock (or Lock visible for all the computers) or Unlock.
 - Force Student Logout: clicks Log out (or Log out visible for all) and confirms.

- Restart Workstation or Shut Down Workstation: clicks Restart or Shutdown and confirms.
- 3. The system sends the command to the student's computer.
- 4. The computer does the command, and the page shows the result.

Exception Scenario:

- If the computer is not connected:
 - The system does not send the command.
- If more than 100 computers are chosen at once:
 - The system does not send the command.
- If the session rule does not allow remote control, or the session has ended:
 - The system does not start remote control.

Additional Remarks:

- Start Remote Control always includes Send Remote Input.
- The student sees a notice while the teacher is in control.
- A restart happens after 10 seconds and a shutdown after 15 seconds.
Unsaved work is lost.

Send Message to Student

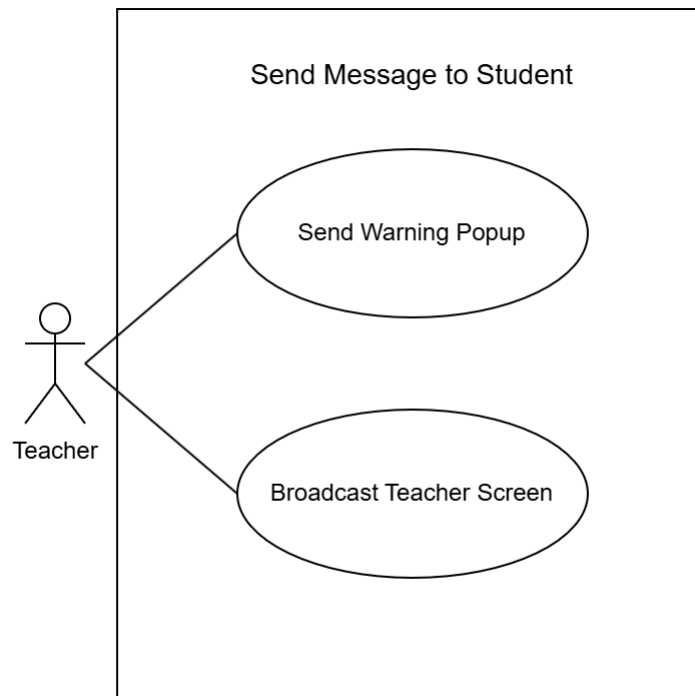


Figure 3.21: System Use Case for Send Message to Student

Use Case Name: Send Message to Student

Purpose: To allow the teacher to send a warning to students, or show the teacher's screen on all the student computers.

Actors: Teacher

Input Parameters:

- Warning title and message, and one student or all
- The teacher's screen to share, for a broadcast

Output Parameters: A warning on the student screens, or the teacher's screen on every student computer

Pre-Condition:

- The teacher is logged in.
- Students are logged in on the lab computers.

Post-Condition: The students see the warning until they close it, or see the teacher's screen until the broadcast stops.

Successful Scenario:

1. The teacher opens the Live Monitoring page.
2. The teacher chooses what to do:
 - Send Warning Popup: clicks Send Warning on a student (or Warn all), types the title and message, and sends it.
 - Broadcast Teacher Screen: clicks Broadcast screen, picks the screen to share, and later clicks Stop Broadcast.
3. The system sends it to the student computers.
4. The student computers show the warning or the teacher's screen.

Exception Scenario:

- If the title or message is empty or too long:
 - The system does not send the warning.
- If a screen picture is too large:
 - The system does not send that picture.

Additional Remarks:

- A warning is also saved in the student's notifications.
- A broadcast is useful for showing a lesson to the whole class.

Manage Monitoring Alert

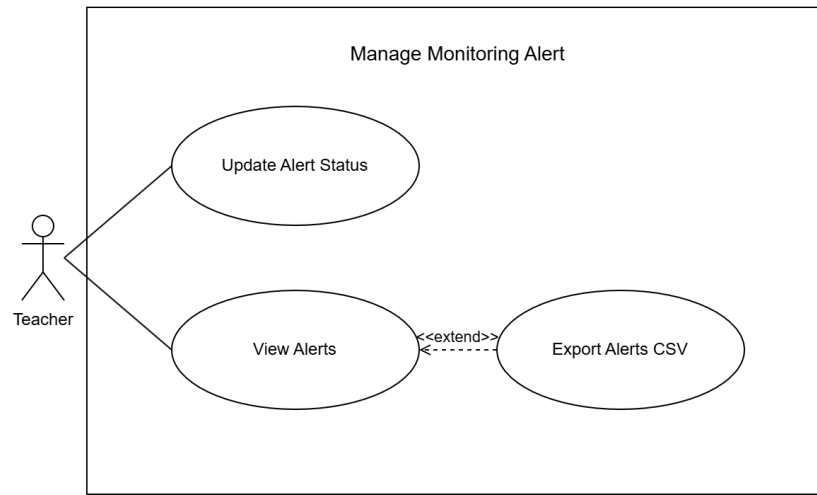


Figure 3.22: System Use Case for Manage Monitoring Alert

Use Case Name: Manage Monitoring Alert

Purpose: To allow the teacher to see the alerts about their students, such as blocked websites, mark them as seen, dismissed or open again, and download them.

Actors: Teacher

Input Parameters:

- Filters: date, student and status (optional)
- The alerts and their new status, with a reason when dismissing

Output Parameters:

- The Alerts page with each alert's status
- A CSV file, when the teacher exports

Pre-Condition: The teacher is logged in.

Post-Condition: The alert status and the open-alert count are updated, or the CSV file is downloaded.

Successful Scenario:

1. The teacher opens the Alerts page and sets the filters (optional).
2. The system shows the alerts (View Alerts).
3. The teacher chooses what to do:
 - Update Alert Status: selects alerts and clicks Acknowledge, Dismiss or Reopen.
 - Export Alerts CSV: clicks Export CSV.
4. The system saves the new status, or makes the CSV file and the browser downloads it.

Exception Scenario:

- If no alert is selected:
 - Nothing changes.
- If nothing matches the filters:
 - The list is empty, and the CSV file has only the column names.

Additional Remarks:

- Export Alerts CSV extends View Alerts: it is optional.
- The system saves who changed an alert and when.

Export Teacher Records

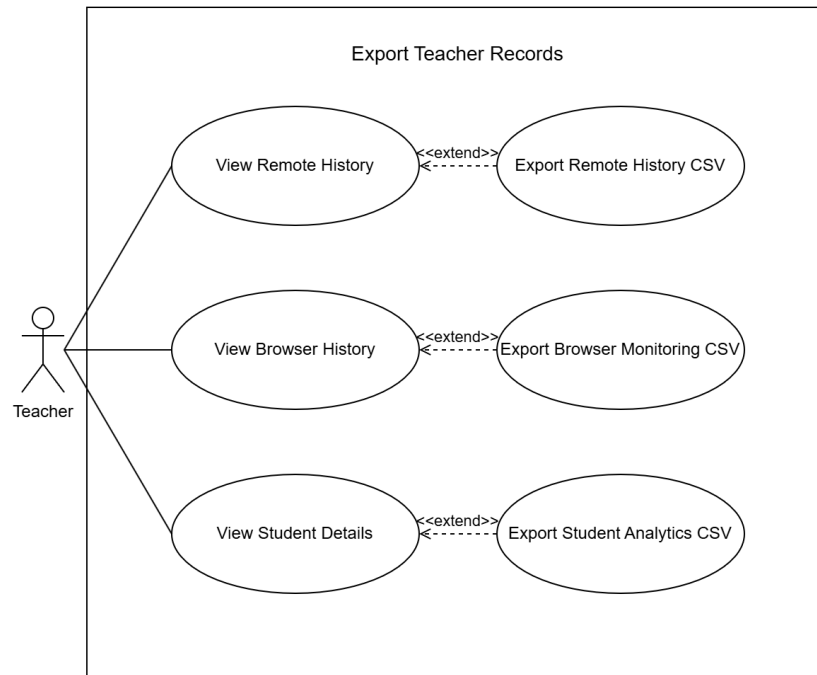


Figure 3.23: System Use Case for Export Teacher Records

Use Case Name: Export Teacher Records

Purpose: To allow the teacher to see the remote commands sent, the websites the students opened, and one student's activity, and to download them as CSV files.

Actors: Teacher

Input Parameters: Filters: date, command, student or browser (optional)

Output Parameters:

- The Remote History, Browser History or student details page
- A CSV file, when the teacher exports

Pre-Condition: The teacher is logged in.

Post-Condition: The teacher sees the records, and has the CSV file if one was exported.
Nothing is changed.

Successful Scenario:

1. The teacher opens the Remote History page (View Remote History), the Browser History page (View Browser History) or a student's details page (View Student Details).
2. The teacher sets the filters (optional).
3. The system shows the records.
4. If the teacher needs a file, the teacher clicks the export button on the page:
 - Export Remote History CSV, Export Browser Monitoring CSV or Export Student Analytics CSV.
5. The system makes the CSV file and the browser downloads it.

Exception Scenario:

- If nothing matches the filters:
 - The page shows an empty list, and the CSV file has only the column names.

Additional Remarks:

- Each export extends its page: it is optional and happens only while the page is open.

STUDENT

Log In at Workstation

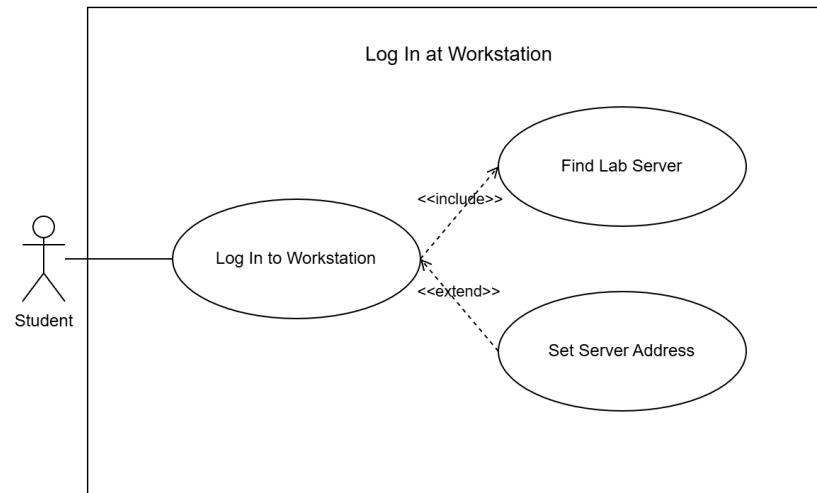


Figure 3.24: System Use Case for Log In at Workstation

Use Case Name: Log In at Workstation

Purpose: To allow the student to log in on a lab computer and join the lab session.

Actors: Student

Input Parameters:

- Username
- Password
- Server address (only when the app cannot find the server)

Output Parameters:

- Login success or error message
- The session screen in the CAMS student app

Pre-Condition:

- The teacher has started the lab session.
- The CAMS student app is installed on the lab computer.

Post-Condition: The student is in the lab session, and the screen is monitored.

Successful Scenario:

1. The student opens the CAMS student app.
2. The app finds the CAMS server on the school network (Find Lab Server).
3. The student enters the username and password and clicks Sign in (Log In to Workstation).
4. The system checks the account:
 - If valid, the system starts the student's session and the app shows the session screen.
 - If not valid, the app shows an error and the student can try again.

Exception Scenario:

- If the app cannot find the server:
 - The student types the address given by the teacher and clicks Save and retry (Set Server Address).
- If no lab session is running:
 - The app tells the student to wait for the teacher.
- If the student enters a wrong password 5 times:
 - The system locks the account for 15 minutes.

Additional Remarks:

- Log In to Workstation always includes Find Lab Server.
- Set Server Address extends Log In to Workstation: it is optional.

Log Out at Workstation

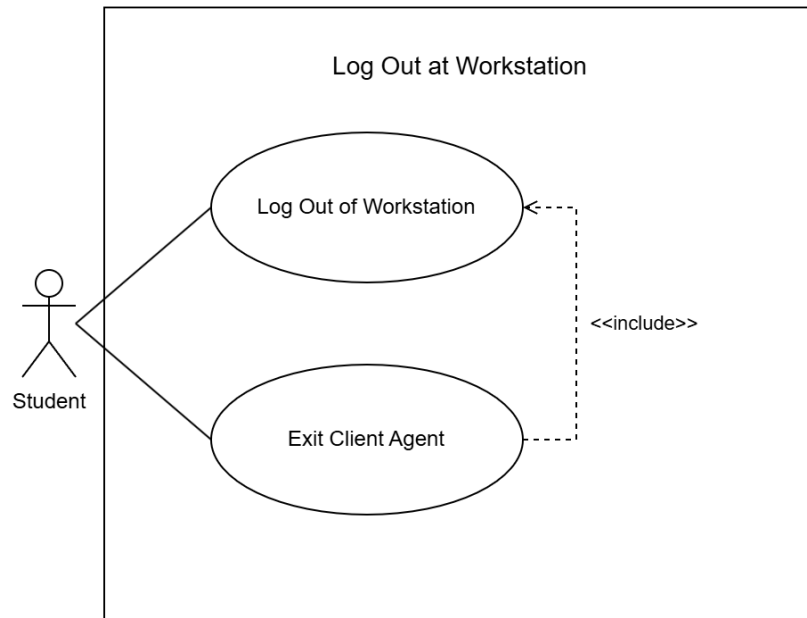


Figure 3.25: System Use Case for Log Out at Workstation

Use Case Name: Log Out at Workstation

Purpose: To allow the student to log out of the lab computer, or close the CAMS student app.

Actors: Student

Input Parameters: None (the student clicks Sign out or Exit)

Output Parameters: The login screen, or the app closes

Pre-Condition: The student is logged in.

Post-Condition: The student's session ends and the computer is free for the next student.

Successful Scenario:

1. The student chooses what to do:
 - Log Out of Workstation: clicks Sign out in the app.

- Exit Client Agent: right-clicks the CAMS icon and clicks Exit.
2. The system ends the student's session.
 3. The app shows the login screen, or closes if the student chose Exit.

Exception Scenario:

- If the server cannot be reached:
 - The app still closes, and the teacher can end the session instead.

Additional Remarks:

- Exit Client Agent always includes Log Out of Workstation.

Change Password at Workstation

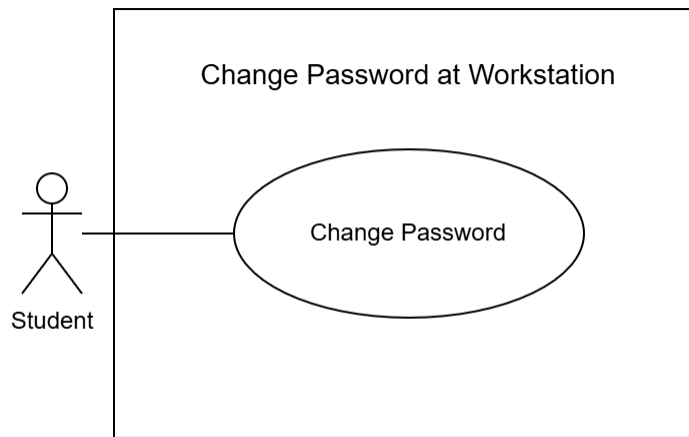


Figure 3.26: System Use Case for Change Password at Workstation

Use Case Name: Change Password at Workstation

Purpose: To allow the student to change their own password in the CAMS student app.

Actors: Student

Input Parameters:

- Current password
- New password, typed twice

Output Parameters: A message that the password was changed

Pre-Condition: The student is logged in on a lab computer.

Post-Condition: The new password is saved and must be used at the next login.

Successful Scenario:

1. The student clicks Change password in the app (Change Password).
2. The student enters the current password and the new password twice.

3. The system checks the passwords:

- If correct, the system saves the new password and shows a message.

Exception Scenario:

- If the current password is wrong:
 - The system shows an error.
- If the new password has fewer than 8 characters, or is the same as the old one:
 - The system asks the student to enter it again.
- If there are 5 wrong tries:
 - The student must wait a minute before trying again.

Additional Remarks:

- Students change their password only in the app, not on the website.